



Query Processing

Database System Concepts, 6th Ed.

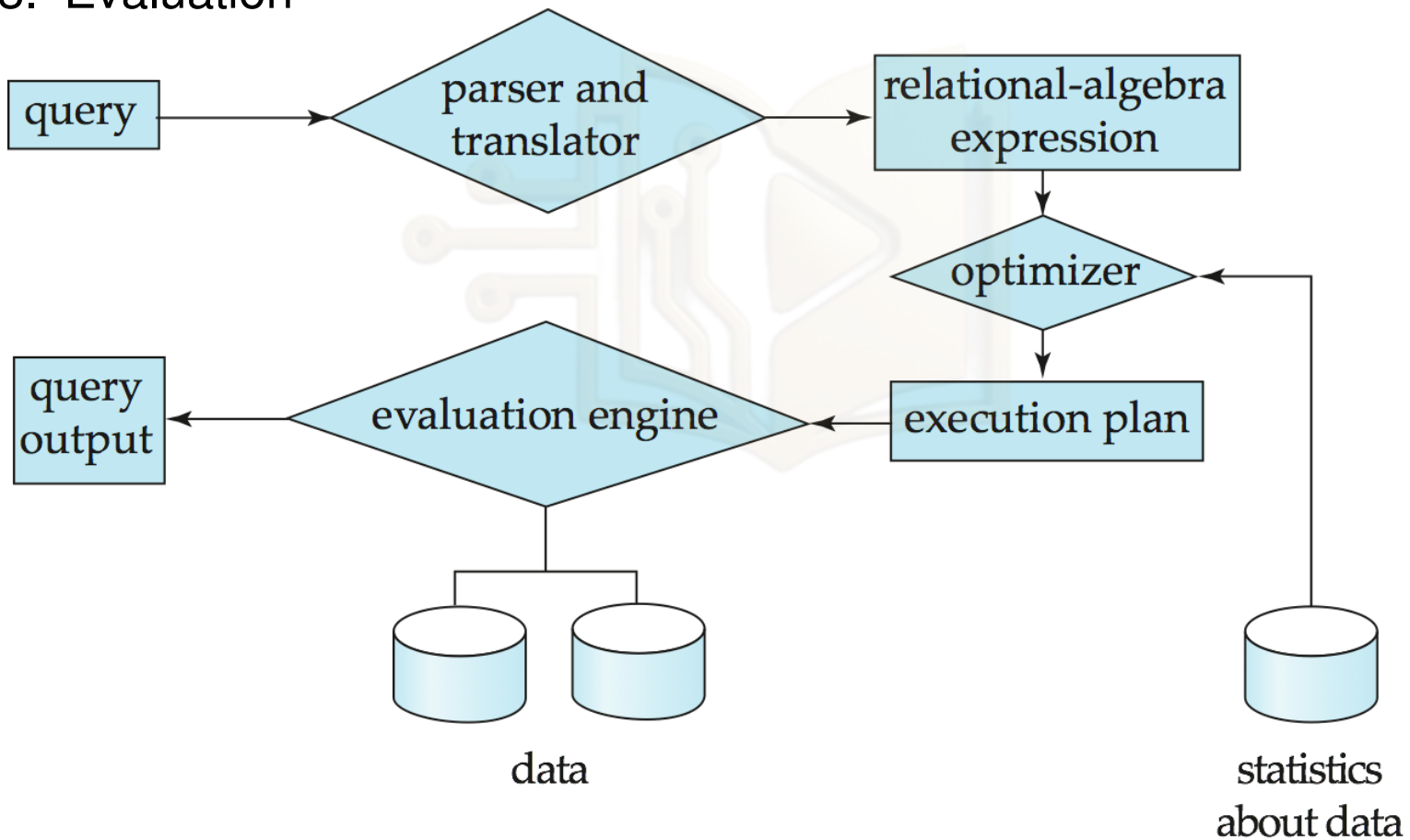
©Silberschatz, Korth and Sudarshan

See www.db-book.com for conditions on re-use



Basic Steps in Query Processing

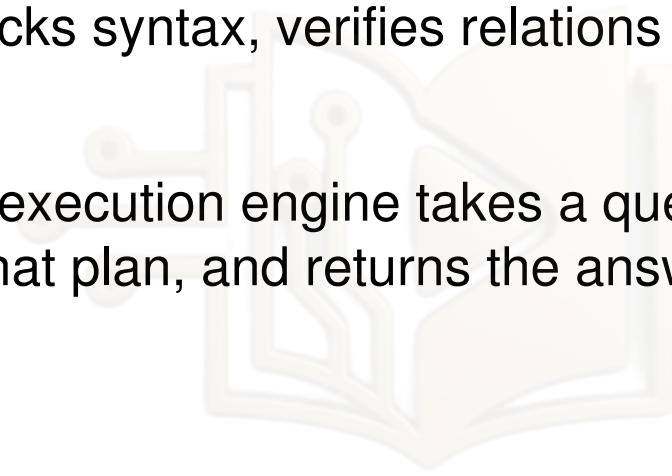
1. Parsing and translation
2. Optimization
3. Evaluation





Basic Steps in Query Processing (Cont.)

- Parsing and translation
 - translate the query into its internal form. This is then translated into relational algebra.
 - Parser checks syntax, verifies relations
- Evaluation
 - The query-execution engine takes a query-evaluation plan, executes that plan, and returns the answers to the query.





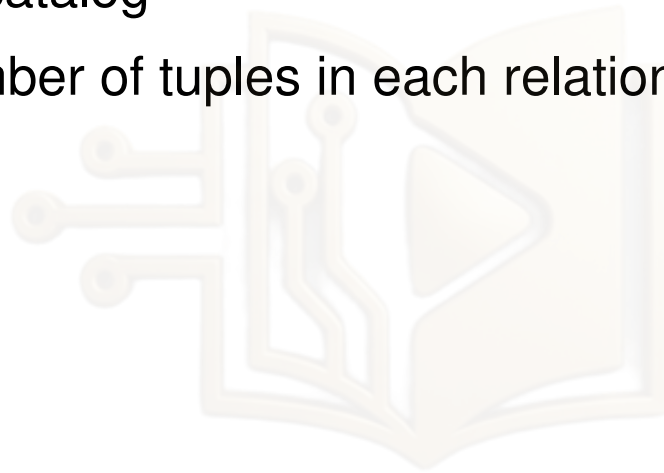
Basic Steps in Query Processing : Optimization

- A relational algebra expression may have many equivalent expressions
 - E.g., $\sigma_{salary < 75000}(\Pi_{salary}(instructor))$ is equivalent to $\Pi_{salary}(\sigma_{salary < 75000}(instructor))$
- Each relational algebra operation can be evaluated using one of several different algorithms
 - Correspondingly, a relational-algebra expression can be evaluated in many ways.
- Annotated expression specifying detailed evaluation strategy is called an **evaluation-plan**.
 - E.g., can use an index on *salary* to find instructors with salary < 75000,
 - or can perform complete relation scan and discard instructors with salary ≥ 75000



Basic Steps: Optimization (Cont.)

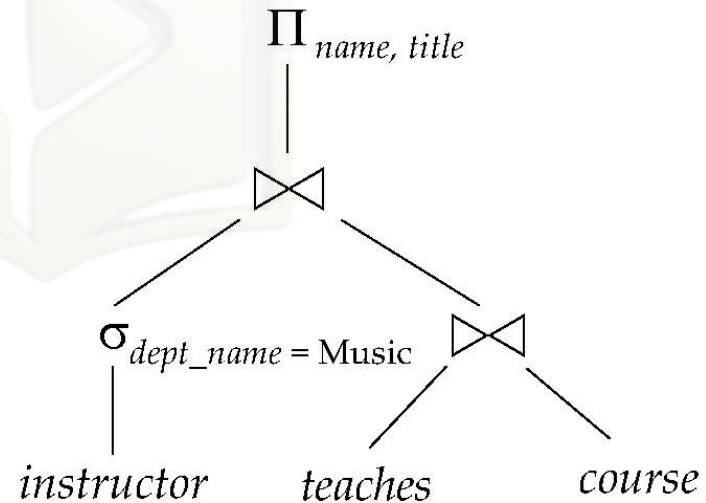
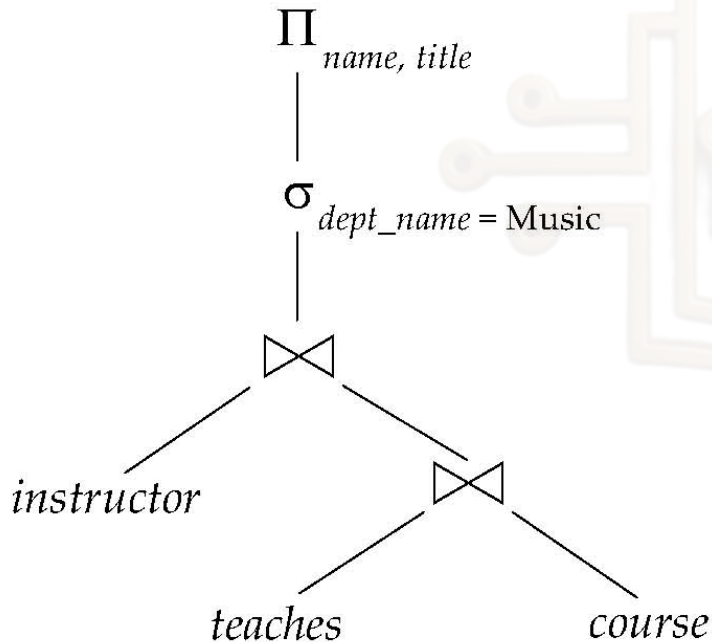
- **Query Optimization:** Amongst all equivalent evaluation plans choose the one with lowest cost.
 - Cost is estimated using statistical information from the database catalog
 - ▶ e.g. number of tuples in each relation, size of tuples, etc.





Query Optimization

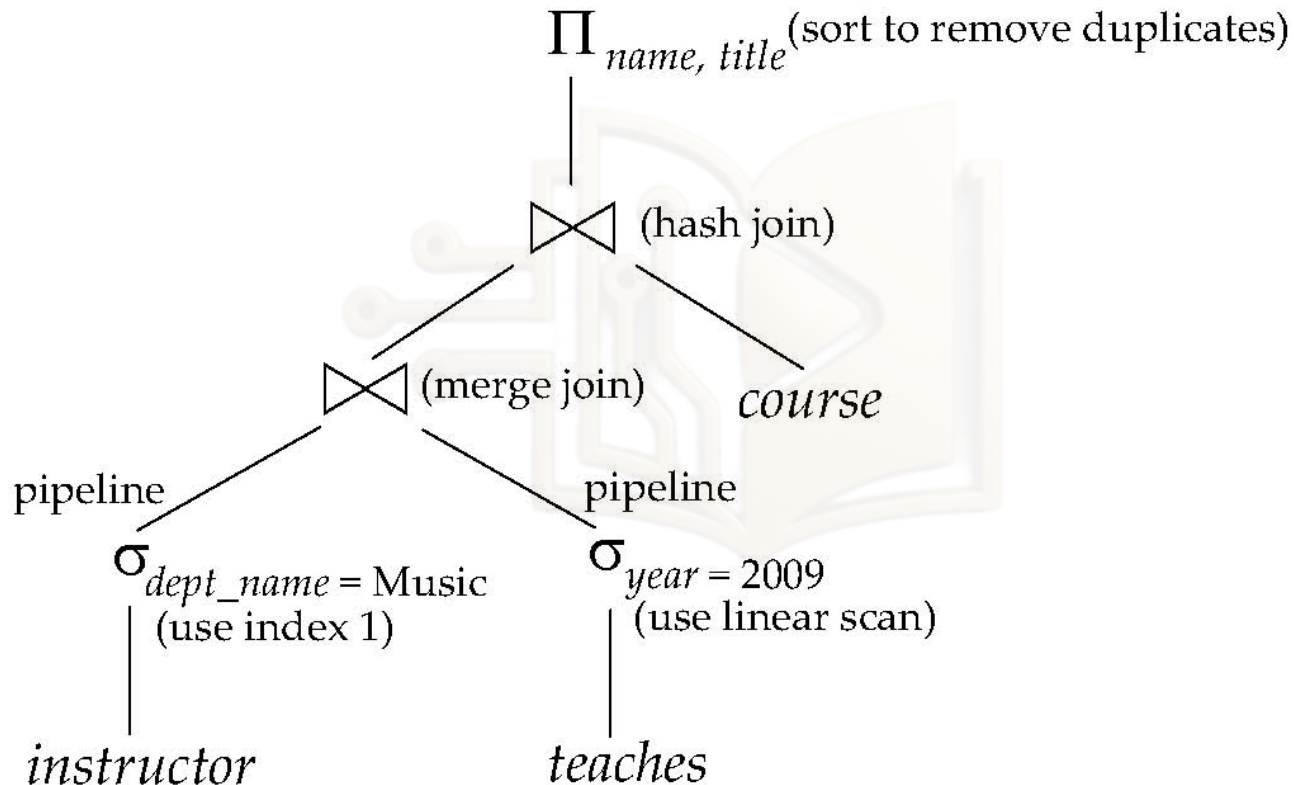
- Alternative ways of evaluating a given query
 - Equivalent expressions
 - Different algorithms for each operation





Query Optimization (Cont.)

- An **evaluation plan** defines exactly what algorithm is used for each operation, and how the execution of the operations is coordinated.



- Find out how to view query execution plans on your favorite database



Query Optimization (Cont.)

- Cost difference between evaluation plans for a query can be enormous
 - E.g. seconds vs. days in some cases
- Steps in **cost-based query optimization**
 1. Generate logically equivalent expressions using **equivalence rules**
 2. Annotate resultant expressions to get alternative query plans
 3. Choose the cheapest plan based on **estimated cost**
- Estimation of plan cost based on:
 - Statistical information about relations. Examples:
 - ▶ number of tuples, number of distinct values for an attribute
 - Statistics estimation for intermediate results
 - ▶ to compute cost of complex expressions
 - Cost formulae for algorithms, computed using statistics



Generating Equivalent Expressions

Database System Concepts, 6th Ed.

©Silberschatz, Korth and Sudarshan

See www.db-book.com for conditions on re-use



Transformation of Relational Expressions

- Two relational algebra expressions are said to be **equivalent** if the two expressions generate the same set of tuples on every *legal* database instance
 - Note: order of tuples is irrelevant
 - we don't care if they generate different results on databases that violate integrity constraints
- In SQL, inputs and outputs are multisets of tuples
 - Two expressions in the multiset version of the relational algebra are said to be equivalent if the two expressions generate the same multiset of tuples on every legal database instance.
- An **equivalence rule** says that expressions of two forms are equivalent
 - Can replace expression of first form by second, or vice versa



Equivalence Rules

1. Conjunctive selection operations can be deconstructed into a sequence of individual selections.

$$\sigma_{\theta_1 \wedge \theta_2}(E) = \sigma_{\theta_1}(\sigma_{\theta_2}(E))$$

2. Selection operations are commutative.

$$\sigma_{\theta_1}(\sigma_{\theta_2}(E)) = \sigma_{\theta_2}(\sigma_{\theta_1}(E))$$

3. Only the last in a sequence of projection operations is needed, the others can be omitted.

$$\Pi_{L_1}(\Pi_{L_2}(\dots(\Pi_{L_n}(E))\dots)) = \Pi_{L_1}(E)$$

4. Selections can be combined with Cartesian products and theta joins.

$\alpha.$ $\sigma_{\theta}(E_1 \times E_2) = E_1 \bowtie_{\theta} E_2$

$\beta.$ $\sigma_{\theta_1}(E_1 \bowtie_{\theta_2} E_2) = E_1 \bowtie_{\theta_1 \wedge \theta_2} E_2$



Equivalence Rules (Cont.)

5. Theta-join operations (and natural joins) are commutative.

$$E_1 \bowtie_{\theta} E_2 = E_2 \bowtie_{\theta} E_1$$

6. (a) Natural join operations are associative:

$$(E_1 \bowtie E_2) \bowtie E_3 = E_1 \bowtie (E_2 \bowtie E_3)$$

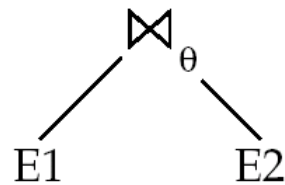
- (b) Theta joins are associative in the following manner:

$$(E_1 \bowtie_{\theta_1} E_2) \bowtie_{\theta_2 \wedge \theta_3} E_3 = E_1 \bowtie_{\theta_1 \wedge \theta_3} (E_2 \bowtie_{\theta_2} E_3)$$

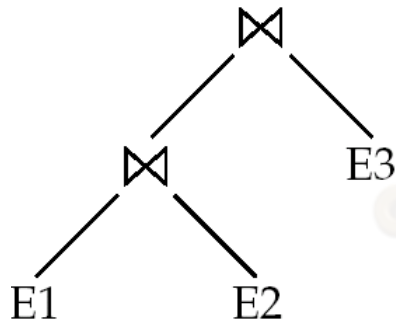
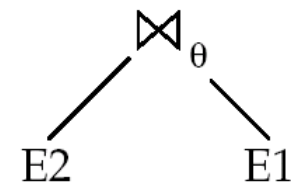
where θ_2 involves attributes from only E_2 and E_3 .



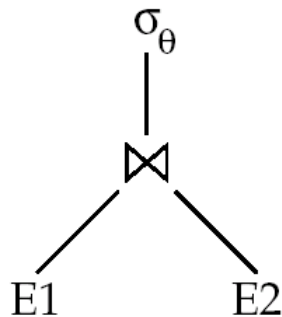
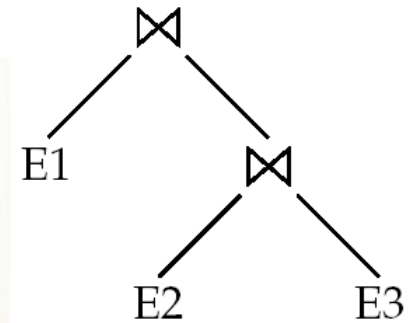
Pictorial Depiction of Equivalence Rules



Rule 5
↔

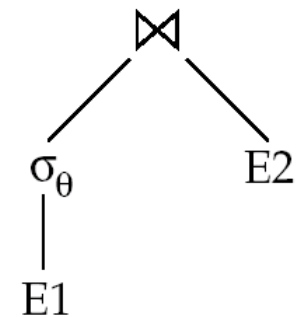


Rule 6a
↔



Rule 7a
↔

If θ only has
attributes from E1





Equivalence Rules (Cont.)

7. The selection operation distributes over the theta join operation under the following two conditions:
- (a) When all the attributes in θ_0 involve only the attributes of one of the expressions (E_1) being joined.

$$\sigma_{\theta_0}(E_1 \bowtie_{\theta} E_2) = (\sigma_{\theta_0}(E_1)) \bowtie_{\theta} E_2$$

- (b) When θ_1 involves only the attributes of E_1 and θ_2 involves only the attributes of E_2 .

$$\sigma_{\theta_1 \wedge \theta_2}(E_1 \bowtie_{\theta} E_2) = (\sigma_{\theta_1}(E_1)) \bowtie_{\theta} (\sigma_{\theta_2}(E_2))$$



Equivalence Rules (Cont.)

8. The projection operation distributes over the theta join operation as follows:

(a) if θ involves only attributes from $L_1 \cup L_2$:

$$\Pi_{L_1 \cup L_2} (E_1 \bowtie_{\theta} E_2) = (\Pi_{L_1} (E_1)) \bowtie_{\theta} (\Pi_{L_2} (E_2))$$

(b) Consider a join $E_1 \bowtie_{\theta} E_2$.

- Let L_1 and L_2 be sets of attributes from E_1 and E_2 , respectively.
- Let L_3 be attributes of E_1 that are involved in join condition θ , but are not in $L_1 \cup L_2$, and
- let L_4 be attributes of E_2 that are involved in join condition θ , but are not in $L_1 \cup L_2$.

$$\Pi_{L_1 \cup L_2} (E_1 \bowtie_{\theta} E_2) = \Pi_{L_1 \cup L_2} ((\Pi_{L_1 \cup L_3} (E_1)) \bowtie_{\theta} (\Pi_{L_2 \cup L_4} (E_2)))$$



Equivalence Rules (Cont.)

9. The set operations union and intersection are commutative

$$E_1 \cup E_2 = E_2 \cup E_1$$

$$E_1 \cap E_2 = E_2 \cap E_1$$

■ (set difference is not commutative).

10. Set union and intersection are associative.

$$(E_1 \cup E_2) \cup E_3 = E_1 \cup (E_2 \cup E_3)$$

$$(E_1 \cap E_2) \cap E_3 = E_1 \cap (E_2 \cap E_3)$$

11. The selection operation distributes over $\cup$, $\cap$ and $-$.

$$\sigma_{\theta} (E_1 - E_2) = \sigma_{\theta} (E_1) - \sigma_{\theta}(E_2)$$

and similarly for $\cup$ and $\cap$ in place of $-$

Also:
$$\sigma_{\theta} (E_1 - E_2) = \sigma_{\theta}(E_1) - E_2$$

and similarly for $\cap$ in place of $-$, but not for $\cup$

12. The projection operation distributes over union

$$\Pi_L(E_1 \cup E_2) = (\Pi_L(E_1)) \cup (\Pi_L(E_2))$$



Transformation Example: Pushing Selections

- Query: Find the names of all instructors in the Music department, along with the titles of the courses that they teach

- $\Pi_{name, title}(\sigma_{dept_name = \text{"Music"}}(instructor \bowtie teaches \bowtie \Pi_{course_id, title}(course)))$

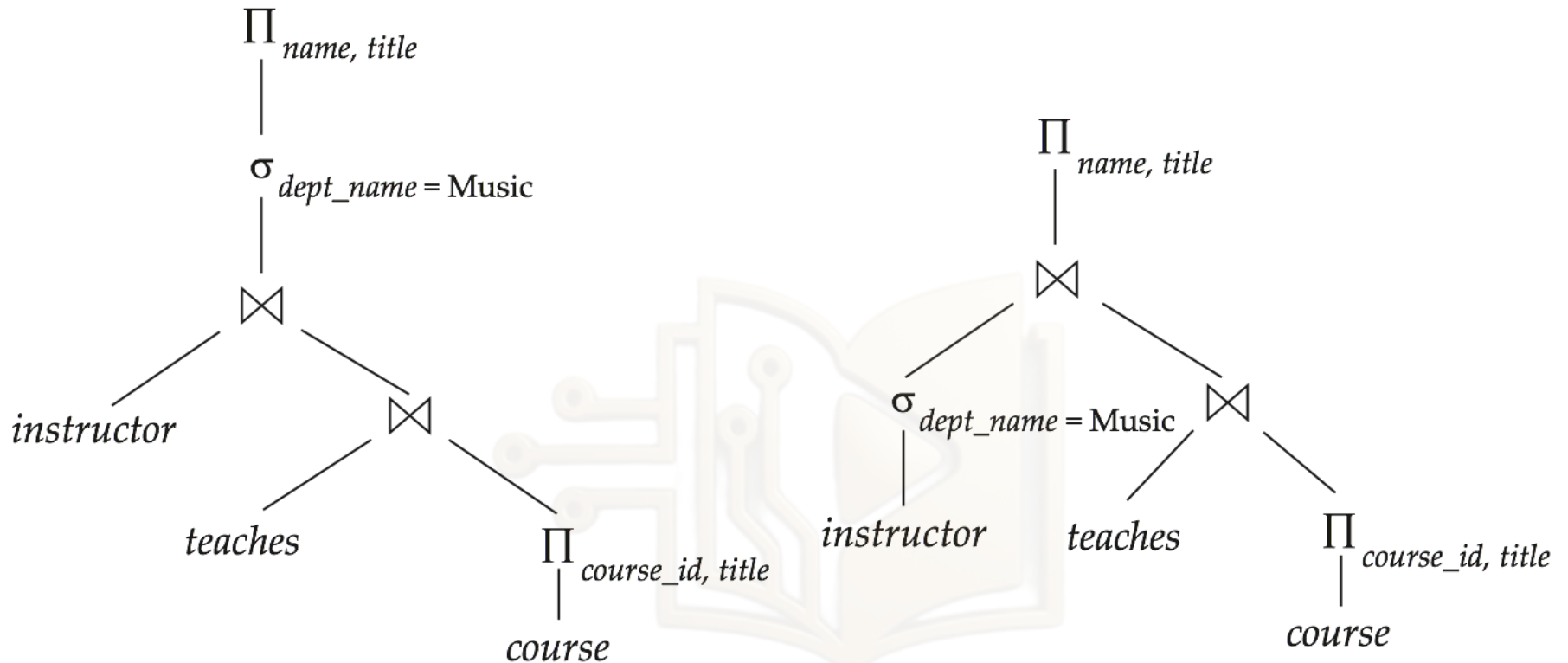
- Transformation using rule 7a.

- $\Pi_{name, title}((\sigma_{dept_name = \text{"Music"}}(instructor)) \bowtie (teaches \bowtie \Pi_{course_id, title}(course)))$

- Performing the selection as early as possible reduces the size of the relation to be joined.



Figure 13.01





Example with Multiple Transformations

- Query: Find the names of all instructors in the Music department who have taught a course in 2009, along with the titles of the courses that they taught

- $\Pi_{name, title}(\sigma_{dept_name = \text{"Music"} \wedge year = 2009}$
 $(instructor \bowtie (teaches \bowtie \Pi_{course_id, title}(course))))$

- Transformation using join associatively (Rule 6a):

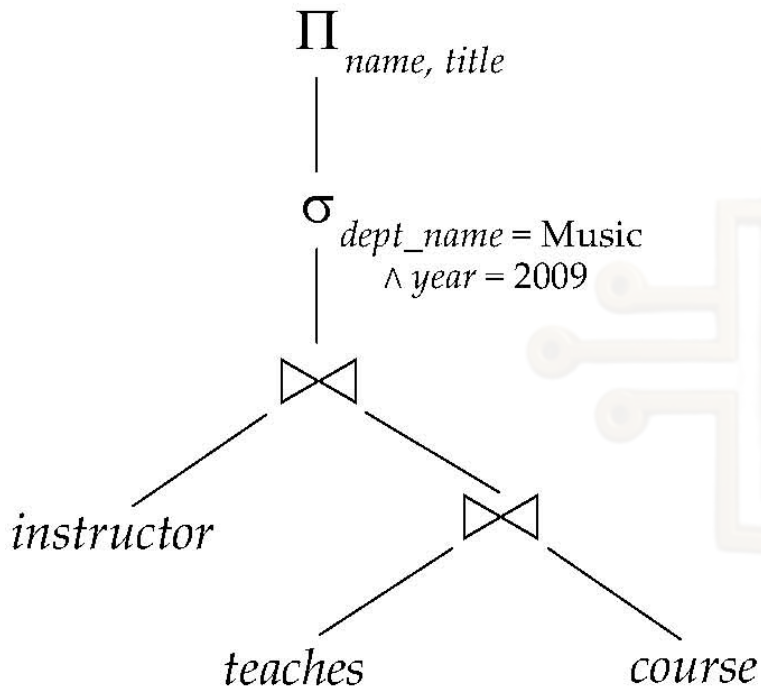
- $\Pi_{name, title}(\sigma_{dept_name = \text{"Music"} \wedge year = 2009}$
 $((instructor \bowtie teaches) \bowtie \Pi_{course_id, title}(course)))$

- Second form provides an opportunity to apply the “perform selections early” rule, resulting in the subexpression

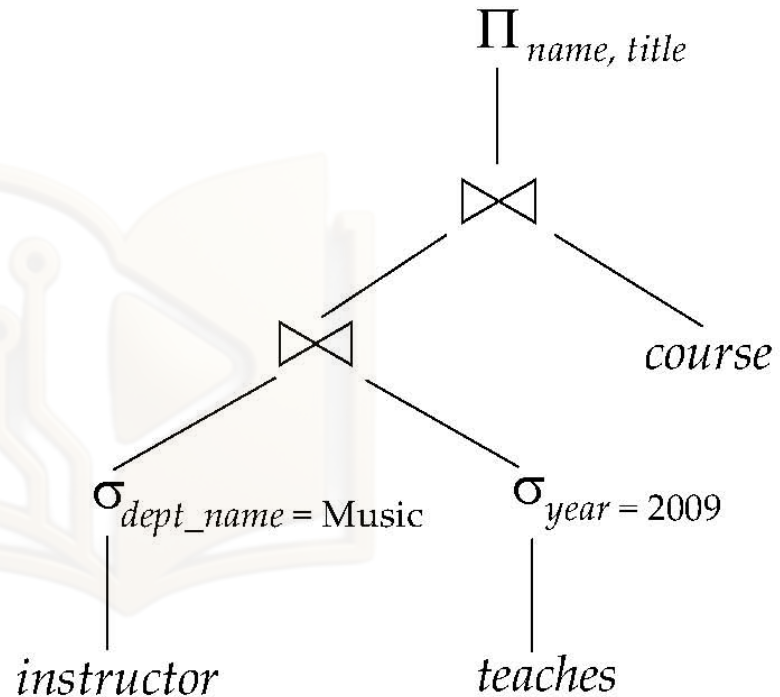
$$\sigma_{dept_name = \text{"Music"}}(instructor) \bowtie \sigma_{year = 2009}(teaches)$$



Multiple Transformations (Cont.)



(a) Initial expression tree



(b) Tree after multiple transformations





Transformation Example: Pushing Projections

- Consider: $\Pi_{name, title}(\sigma_{dept_name = \text{"Music"}}(instructor \bowtie teaches) \bowtie \Pi_{course_id, title}(course))$

- When we compute

$$(\sigma_{dept_name = \text{"Music"}}(instructor \bowtie teaches))$$

we obtain a relation whose schema is:

$(ID, name, dept_name, salary, course_id, sec_id, semester, year)$

- Push projections using equivalence rules 8a and 8b; eliminate unneeded attributes from intermediate results to get:

$$\Pi_{name, title}(\Pi_{name, course_id}(\sigma_{dept_name = \text{"Music"}}(instructor \bowtie teaches) \bowtie \Pi_{course_id, title}(course)))$$

- Performing the projection as early as possible reduces the size of the relation to be joined.



Join Ordering Example

- For all relations r_1 , r_2 , and r_3 ,

$$(r_1 \bowtie r_2) \bowtie r_3 = r_1 \bowtie (r_2 \bowtie r_3)$$

(Join Associativity)

- If $r_2 \bowtie r_3$ is quite large and $r_1 \bowtie r_2$ is small, we choose

$$(r_1 \bowtie r_2) \bowtie r_3$$

so that we compute and store a smaller temporary relation.



Join Ordering Example (Cont.)

- Consider the expression

$$\Pi_{name, title}(\sigma_{dept_name = \text{"Music"}}(instructor) \bowtie teaches \bowtie \Pi_{course_id, title}(course)))$$

- Could compute $teaches \bowtie \Pi_{course_id, title}(course)$ first, and join result with

$$\sigma_{dept_name = \text{"Music"}}(instructor)$$

but the result of the first join is likely to be a large relation.

- Only a small fraction of the university's instructors are likely to be from the Music department
 - it is better to compute

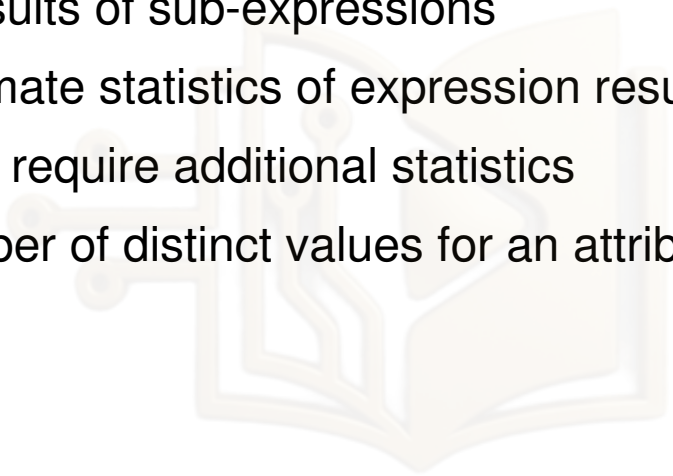
$$\sigma_{dept_name = \text{"Music"}}(instructor) \bowtie teaches$$

first.



Cost Estimation

- Cost of each operator computed
 - Need statistics of input relations
 - ▶ E.g. number of tuples, sizes of tuples
- Inputs can be results of sub-expressions
 - Need to estimate statistics of expression results
 - To do so, we require additional statistics
 - ▶ E.g. number of distinct values for an attribute





Heuristic Optimization

- Cost-based optimization is expensive, even with dynamic programming.
- Systems may use *heuristics* to reduce the number of choices that must be made in a cost-based fashion.
- Heuristic optimization transforms the query-tree by using a set of rules that typically (but not in all cases) improve execution performance:
 - Perform selection early (reduces the number of tuples)
 - Perform projection early (reduces the number of attributes)
 - Perform most restrictive selection and join operations (i.e. with smallest result size) before other similar operations.
 - Some systems use only heuristics, others combine heuristics with partial cost-based optimization.



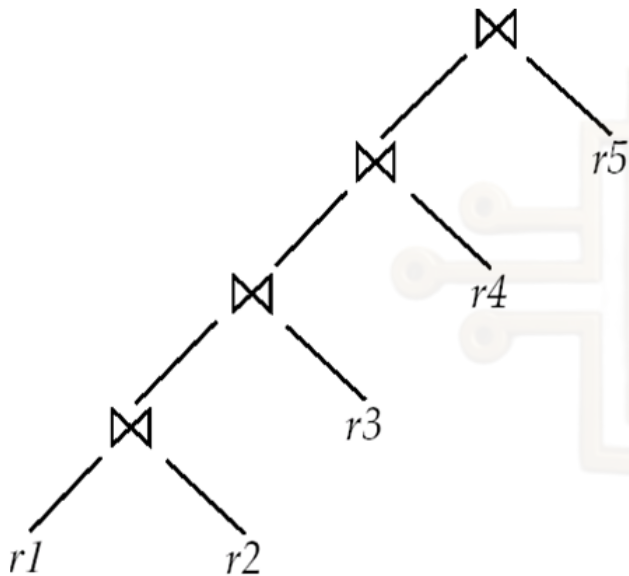
Choice of Evaluation Plans

- Must consider the interaction of evaluation techniques when choosing evaluation plans
 - choosing the cheapest algorithm for each operation independently may not yield best overall algorithm. E.g.
 - ▶ merge-join may be costlier than hash-join, but may provide a sorted output which reduces the cost for an outer level aggregation.
 - ▶ nested-loop join may provide opportunity for pipelining
- Practical query optimizers incorporate elements of the following two broad approaches:
 1. Search all the plans and choose the best plan in a cost-based fashion.
 2. Uses heuristics to choose a plan.

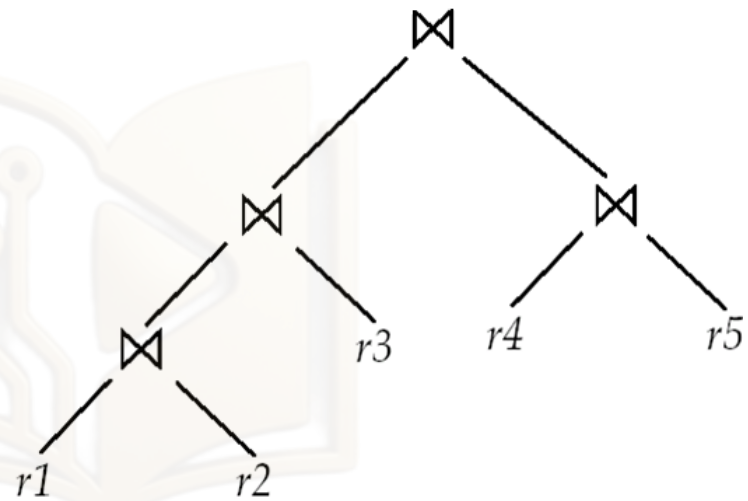


Left Deep Join Trees

- In **left-deep join trees**, the right-hand-side input for each join is a relation, not the result of an intermediate join.



(a) Left-deep join tree



(b) Non-left-deep join tree



Materialized Views**

- A **materialized view** is a view whose contents are computed and stored.
- Consider the view
create view *department_total_salary*(*dept_name*, *total_salary*) **as**
select *dept_name*, **sum**(*salary*)
from *instructor*
group by *dept_name*
- Materializing the above view would be very useful if the total salary by department is required frequently
 - Saves the effort of finding multiple tuples and adding up their amounts



Materialized View Maintenance

- The task of keeping a materialized view up-to-date with the underlying data is known as **materialized view maintenance**
- Materialized views can be maintained by recomputation on every update
- A better option is to use **incremental view maintenance**
 - **Changes to database relations are used to compute changes to the materialized view, which is then updated**
- View maintenance can be done by
 - Manually defining triggers on insert, delete, and update of each relation in the view definition
 - Manually written code to update the view whenever database relations are updated
 - Periodic recomputation (e.g. nightly)
 - Above methods are directly supported by many database systems
 - ▶ Avoids manual effort/correctness issues



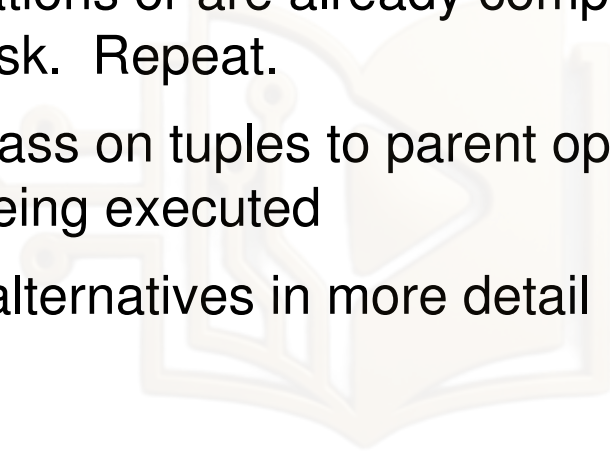
Materialized View Selection

- **Materialized view selection**: “What is the best set of views to materialize?”.
- **Index selection**: “what is the best set of indices to create”
 - closely related, to materialized view selection
 - ▶ but simpler
- Materialized view selection and index selection based on typical system **workload** (queries and updates)
 - Typical goal: minimize time to execute workload , subject to constraints on space and time taken for some critical queries/updates
 - One of the steps in database tuning
- Commercial database systems provide tools (called “tuning assistants” or “wizards”) to help the database administrator choose what indices and materialized views to create



Evaluation of Expressions

- So far: we have seen algorithms for individual operations
- Alternatives for evaluating an entire expression tree
 - **Materialization**: generate results of an expression whose inputs are relations or are already computed, **materialize** (store) it on disk. Repeat.
 - **Pipelining**: pass on tuples to parent operations even as an operation is being executed
- We study above alternatives in more detail



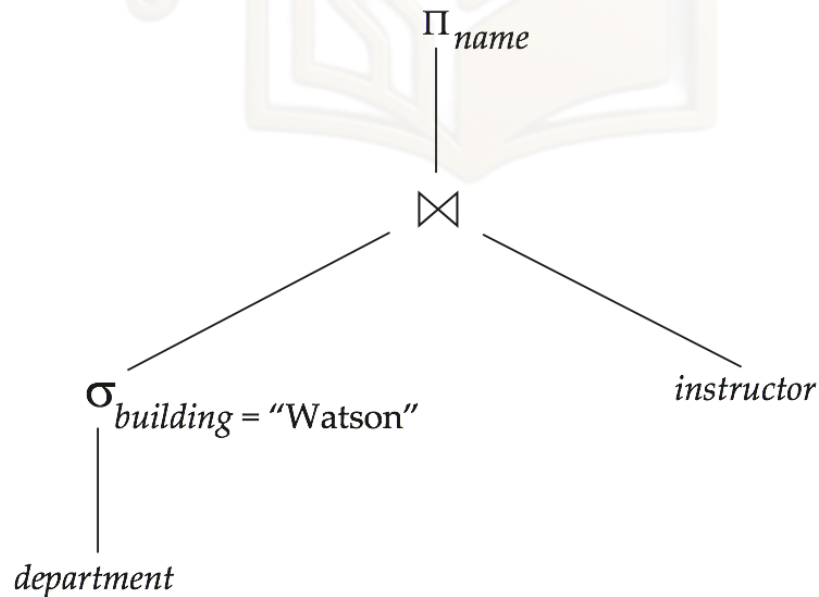


Materialization

- **Materialized evaluation:** evaluate one operation at a time, starting at the lowest-level. Use intermediate results materialized into temporary relations to evaluate next-level operations.
- E.g., in figure below, compute and store

$\sigma_{building="Watson"}(department)$

then compute the store its join with *instructor*, and finally compute the projection on *name*.





Materialization (Cont.)

- Materialized evaluation is always applicable
- Cost of writing results to disk and reading them back can be quite high
 - Our cost formulas for operations ignore cost of writing results to disk, so
 - ▶ Overall cost = Sum of costs of individual operations + cost of writing intermediate results to disk
- **Double buffering**: use two output buffers for each operation, when one is full write it to disk while the other is getting filled
 - Allows overlap of disk writes with computation and reduces execution time



Pipelining

- **Pipelined evaluation** : evaluate several operations simultaneously, passing the results of one operation on to the next.
- E.g., in previous expression tree, don't store result of

$$\sigma_{building="Watson"}(department)$$

- instead, pass tuples directly to the join.. Similarly, don't store result of join, pass tuples directly to projection.
- Much cheaper than materialization: no need to store a temporary relation to disk.
- Pipelining may not always be possible – e.g., sort, hash-join.
- For pipelining to be effective, use evaluation algorithms that generate output tuples even as tuples are received for inputs to the operation.
- Pipelines can be executed in two ways: **demand driven** and **producer driven**



Pipelining (Cont.)

- In **demand driven** or **lazy** evaluation
 - system repeatedly requests next tuple from top level operation
 - Each operation requests next tuple from children operations as required, in order to output its next tuple
 - In between calls, operation has to maintain “**state**” so it knows what to return next
- In **producer-driven** or **eager** pipelining
 - Operators produce tuples eagerly and pass them up to their parents
 - ▶ Buffer maintained between operators, child puts tuples in buffer, parent removes tuples from buffer
 - ▶ if buffer is full, child waits till there is space in the buffer, and then generates more tuples
 - System schedules operations that have space in output buffer and can process more input tuples
- Alternative name: **pull** and **push** models of pipelining